Question 1

Rank the following complexities in terms of their rate of growth starting with slowest to fastest: O(n), O(log n), O(n^5), O(n log n), O(1).

Answer:

- Slowest: O(1)
- Next: O(log n)
- Next: O(n)
- Next: O(n log n)
- Fastest: O(n^5)

Question 2

Find the Big-O complexity for the following functions:

a) $4n^2 + 20n + 9$ → **$O(n^2)$** b) $25n^3 + 15$ → **$O(n^3)$** c) $n^3 + 5\log n + 6$ → **$O(n^3)$** d) $105 + 105n + 105n^2$ → **$O(n^2)$** e) $72n\ (n + 3n^2) = 72n^2 + 216n^3$ → **$O(n^3)$** f) $28 + 324n - 5$ → **O(n)**

Question 3

Design an algorithm that runs in worst-case O(n) time to add all the numbers from 1 to a given number n (including n).

Answer (Algorithm):

Code

```
Algorithm Sum_Natural_Numbers(n):
  total ← 0
  for i ← 1 to n do
     total ← total + i
  endfor
  return total
```

Complexity: O(n) because the loop runs n times.

Question 4

For each code fragment, give the worst-case running time complexity:

a)

Code

```
x = 4
for i := 0 to number do:
  x := x+1
```

```
endfor
```

Complexity: **O(number)** → O(n).

b)

Code

```
choice := get_choice()
if choice=1 then:
  print "Ready"
elseif choice=2 then:
  for i := 0 to cycles do:
    print i
  endfor
endif
```

Complexity: **O(cycles)** in worst case.

c)

Code

```
increase(val, iter):
  for i := 1 to iter do:
    val := val+5
  return val


calculate(val, iter):
  for i := 1 to iter do:
    val := val+increase(val, iter)
  return val
```

- increase → O(iter)
- calculate → iter calls to increase → O(iter × iter) = **O(iter^2)**.

d)

Code

```
increase(val, iter):
  for i := 1 to iter do:
    val := val+5
```

```
  return val


calculate(val, iter):
  for i := 1 to iter do:
    val := val+increase(val, i)
  return val
```

- increase → O(i)
- calculate → sum of complexities from i=1 to iter → 1+2+…+iter = O(iter²). So complexity = **O(iter²)**.

Question 5

Fill in the blanks with ≤, ≥, or = :

(i) If f(n) is O(g(n)), then for some constants c and k, for all n ≥ k, f(n) **≤** c g(n).

(ii) If f(n) is Θ(g(n)), then for some constants c and k, for all n ≥ k, f(n) **=** c g(n).

Question 6

Show that 4n + 12 is O(n).

We need constants c and k such that: 4n + 12 ≤ c·n for all n ≥ k. Choose c = 5, k = 12. For n ≥ 12, 4n + 12 ≤ 5n. Therefore, **4n + 12 is O(n).**

Question 7

Show that $5n^2 - 16$ is $O(n^2)$.

We need constants c and k such that: $5n^2 - 16 \leq c \cdot n^2$ for all n ≥ k. Choose c = 5, k = 2. For n ≥ 2, $5n^2 - 16 \leq 5n^2$. Therefore, **$5n^2 - 16$ is $O(n^2)$.**